

# Docker Container Monitoring & Alerting

**Scope:** Self-hosted Linux Docker servers (fe4-docker-01, fe4-docker-02, ...) **Owner:** FE4 Team · **Status:** In production on fe4-docker-01 and fe4-docker-02

This document is **self-contained**: the full script and config are embedded in §4 (Steps 3-4). A new server can be set up by reading this doc alone — no copying files between servers required.

## 1. Why we need this

Our applications (e.g. `fe4-kafkabridge-system`) run as Docker containers on self-hosted Linux servers. Before this system existed:

- If a container crashed (unhandled exception, OOM kill), it **stayed down** — Docker's default restart policy is `no`.
- Nobody was notified. The team had **zero visibility** until someone manually checked or a downstream symptom appeared (e.g. Kafka consumer lag).
- A crash at night could mean **hours of silent downtime**.

This system fixes both problems with two independent layers:

| Layer               | Job                                         | Mechanism                                         |
|---------------------|---------------------------------------------|---------------------------------------------------|
| Layer 1 — Recovery  | bring crashed containers back automatically | Docker restart policy <code>unless-stopped</code> |
| Layer 2 — Detection | tell humans a crash happened                | cron script + email via internal SMTP relay       |

**Design rule:** manual `docker stop` / `docker kill` = **silent** (maintenance is not an incident). Crashes and crash loops = **mail**.

## 2. How it works

### 2.1 Layer 1 — restart policy `unless-stopped`

| Event                                                      | Behavior                                                              |
|------------------------------------------------------------|-----------------------------------------------------------------------|
| App crash / OOM / host-level kill                          | auto-restart within ~1 second                                         |
| Docker daemon restart                                      | container comes back                                                  |
| Server reboot                                              | container comes back (requires <code>systemctl enable docker</code> ) |
| Manual <code>docker stop</code> / <code>docker kill</code> | <b>stays down</b> (operator intent respected)                         |
| Repeated crashes                                           | retries forever with backoff (100ms doubling, capped at 1 min)        |

Key mechanism — **how Docker tells crash from manual stop**: it does not inspect the death signal (a crash SIGKILL and `docker kill` SIGKILL are identical). It tracks whether the stop request went **through its own API**. API stop/kill → "manually stopped" flag set, no restart, RestartCount frozen. Process died without an API request → crash → restart policy fires → **RestartCount +1**.

### 2.2 Layer 2 — monitor script

Runs every 5 minutes via cron. Reads exactly four inputs:

1. `/etc/container-monitor.conf` — what to watch, where to mail
2. `docker info` — is the daemon alive
3. `docker inspect` → `{{.State.Status}}` and `{{.RestartCount}}` per container
4. `/var/lib/container-monitor/<name>.state` — snapshot from previous run  
(`status|count`)

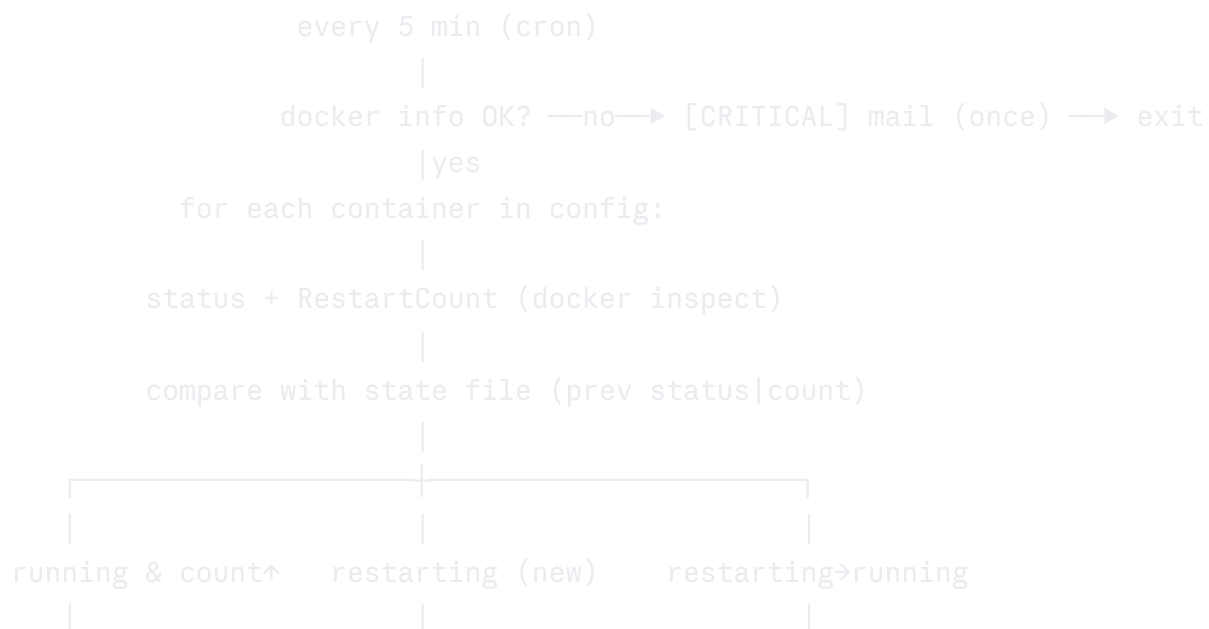
Decisions are transitions between (4) and (3). **Logs are never parsed** — `docker logs --tail 20` is attached to mails purely as human context.

| Current vs previous                            | Meaning                                                           | Action                 |
|------------------------------------------------|-------------------------------------------------------------------|------------------------|
| <code>running</code> , count increased         | crashed & auto-recovered (possibly between checks)                | [CRASH] mail           |
| became <code>restarting</code>                 | crash loop in progress                                            | [DOWN] mail (once)     |
| <code>restarting</code> → <code>running</code> | loop resolved (self-healed or redeployed)                         | [RECOVERED] mail       |
| sits in <code>exited</code>                    | manual stop (only possibility under <code>unless-stopped</code> ) | silent                 |
| <code>missing</code>                           | container removed (mid-deploy)                                    | silent                 |
| <code>docker info</code> fails                 | daemon dead                                                       | [CRITICAL] mail (once) |

#### Reliability details:

- **Transition-based:** container down for 3 days = 1 mail, not 864.
- **Send-or-retry:** state file is only advanced after a successful SMTP send; a failed mail is retried on the next run — events are never lost.
- **flock:** overlapping cron runs impossible.
- **SMTP via curl** with CRLF normalization (see §7 Troubleshooting).

### 2.3 Workflow diagram



```
[CRASH] mail      [DOWN] mail      [RECOVERED] mail
|                |                |
└──────────┬──┐ mail OK? ─┬─ yes → write status|count to state file
              │            └─ no  → keep old state (auto-retry next run)

exited / missing / no change ────> silent, update state file
```

### 3. Prerequisites

- Root access on the target server (`sudo -i`)
- Internal SMTP relay reachable: `smtp.nhorbcomm.internal:587` (no auth required from whitelisted servers; STARTTLS optional)
- Standard tools: `docker`, `curl`, `flock`, `sed` (all present on Ubuntu/Debian)

### 4. Implementation — step by step

All steps as **root** (`sudo -i`).

#### Step 1 — Apply Layer 1 to each application container

```
bash

# check current policy (fresh installs are usually "no")
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' <container-name>

# apply (safe on a running container, no downtime)
docker update --restart unless-stopped <container-name>

# verify
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' <container-name> # unl
```

⚠ **This is a live-container fix only.** Our CI/CD (`deploy-docker-ssh-sad` action) recreates containers on every deploy; the recreated container gets the policy from the `docker run` flags. The `--restart unless-stopped` flag must be present in the deploy action's `docker run` command, otherwise the policy silently regresses to `no` on the next deploy.

#### Step 2 — Docker on boot

```
bash
```

```
systemctl is-enabled docker || systemctl enable docker
```

### Step 3 — Create the config file

```
bash  
  
nano /etc/container-monitor.conf
```

Paste, then edit `CONTAINERS` for **this server's** containers (`docker ps --format '{{.Names}}'` to list them) and set real recipients:

```
bash  
  
#=====br/># container-monitor.conf - configuration for container-monitor.sh  
# Location: /etc/container-monitor.conf (chmod 640, owner root)  
# Plain shell syntax. Arrays are SPACE-separated inside parentheses - NO commas  
#=====br/>  
# Containers to monitor on THIS server  
CONTAINERS=("fe4-kafkabridge-system")  
  
# Alert recipients (space-separated for multiple)  
RCPTS=("team@company.com")  
  
# Sender address  
FROM="no-reply@nhorbcomm.internal"  
  
# SMTP relay  
SMTP="smtp://smtp.nhorbcomm.internal:587"  
  
# "yes" adds --ssl-reqd (STARTTLS) to curl. Relay accepts plain; "no" works.  
SMTP_TLS="no"  
  
# Number of container log lines included in alert mails  
LOG_TAIL=20
```

⚠ Arrays: `("a" "b")` with spaces. A comma becomes part of the value and silently breaks delivery.

### Step 4 — Create the script

```
bash
```

```
nano /opt/monitoring/container-monitor.sh      # (mkdir -p /opt/monitoring first
```

Paste the complete script below **exactly as-is**, with ONE edit: the environment label in the four mail-subject lines is hardcoded as **UAT** — change it to this server's environment (UAT / PROD / QA) before saving.

bash

```

#!/usr/bin/env bash
#=====
# container-monitor.sh - Layer 2 detection for Docker containers
#
# Config : /etc/container-monitor.conf (all settings live there)
# Deploy : /opt/monitoring/container-monitor.sh
# Cron   : */5 * * * * /opt/monitoring/container-monitor.sh >> /var/log/contain
#
# RULES:
#   Crash (single or loop)      -> mail
#   Recovery from crash loop    -> mail
#   Docker daemon down          -> mail (once)
#   Manual docker stop/start    -> SILENT
#   (requires --restart unless-stopped on every monitored container)
#=====
set -u

CONFIG_FILE="/etc/container-monitor.conf"
STATE_DIR="/var/lib/container-monitor"
HOST="$(hostname)"

#----- Load + validate config -----
if [ ! -r "$CONFIG_FILE" ]; then
    echo "$(date -Is) [FATAL] Config not found or unreadable: $CONFIG_FILE" >&2
    exit 1
fi
. "$CONFIG_FILE"

for var in FROM SMTP; do
    [ -n "${!var:-}" ] || { echo "$(date -Is) [FATAL] $var not set in $CONFIG_F
done
[ "${CONTAINERS+set}" = "set" ] && [ "${#CONTAINERS[@]}" -gt 0 ] || { echo "$(d
[ "${RCPTS+set}" = "set" ] && [ "${#RCPTS[@]}" -gt 0 ] || { echo "$(d
LOG_TAIL="${LOG_TAIL:-20}"
SMTP_TLS="${SMTP_TLS:-no}"

mkdir -p "$STATE_DIR"

# Prevent overlapping cron runs
exec 200>"/var/run/container-monitor.lock"
flock -n 200 || exit 0

log() { echo "$(date -Is) [$1] $2"; }

```

```

#----- Mail sender -----
send_mail() { # $1=subject $2=body
    local rcpt_args=() r rc
    for r in "${RCPTS[@]}; do rcpt_args+=(--mail-rcpt "$r"); done

    local tls_args=()
    [ "$SMTP_TLS" = "yes" ] && tls_args=(--ssl-reqd)

    {
        printf 'From: Container Monitor <%s>\n' "$FROM"
        printf 'To: %s\n' "${IFS=,; echo "${RCPTS[*]}"}"
        printf 'Subject: %s\n' "$1"
        printf 'Date: %s\n' "${date -R}"
        printf '\n%s\n' "$2"
    } | sed -e 's/\r$//' -e 's/^\././' -e 's/$/\r/' |
    curl -s --show-error --url "$SMTP" --mail-from "$FROM" \
        "${rcpt_args[@]}" "${tls_args[@]}" --upload-file - --max-time 15
    rc=$?
    [ $rc -eq 0 ] && log INFO "Mail sent: $1" || log ERROR "Mail FAILED (rc=$rc)"
    return $rc
}

#----- Docker daemon check (once, global) -----
DAEMON_STATE="${STATE_DIR}/_daemon.state"
if ! docker info >/dev/null 2>&1; then
    prev=$(cat "$DAEMON_STATE" 2>/dev/null || echo "up")
    if [ "$prev" != "down" ]; then
        send_mail "[CRITICAL] UAT Docker daemon down on ${HOST}" \
"Time : $(date -Is)
Impact : ALL monitored containers unknown, assume DOWN.
Action : ssh ${HOST} && systemctl status docker" \
        && echo "down" > "$DAEMON_STATE"
    fi
    exit 0
fi
echo "up" > "$DAEMON_STATE"

#----- Per-container check -----
check_container() {
    local c="$1"
    local sf="${STATE_DIR}/${c}.state"
    local status count prev_status prev_count subject="" body=""

```



```

status=$(docker inspect -f '{{.State.Status}}' "$c" 2>/dev/null || echo "missing")
count=$(docker inspect -f '{{.RestartCount}}' "$c" 2>/dev/null || echo 0)

if [ -f "$sf" ]; then
    IFS='|' read -r prev_status prev_count < "$sf"
else
    prev_status="running"; prev_count=0
fi

if [ "$status" = "running" ] && [ "$count" -gt "$prev_count" ]; then
    subject="[CRASH] UAT ${c} crashed and auto-recovered on ${HOST}"
    body="Time          : $(date -Is)
RestartCount : ${prev_count} -> ${count}
Status now    : running (recovered by restart policy)
OOMKilled     : $(docker inspect -f '{{.State.OOMKilled}}' "$c" 2>/dev/null)

--- last ${LOG_TAIL} log lines ---
$(docker logs --tail "$LOG_TAIL" "$c" 2>&1)"

    elif [ "$status" = "restarting" ] && [ "$prev_status" != "restarting" ]; then
        subject="[DOWN] UAT ${c} is crash-looping on ${HOST}"
        body="Time          : $(date -Is)
Status      : restarting (Docker retrying with backoff, will not give up)
RestartCount : ${count}
ExitCode     : $(docker inspect -f '{{.State.ExitCode}}' "$c" 2>/dev/null)
OOMKilled     : $(docker inspect -f '{{.State.OOMKilled}}' "$c" 2>/dev/null)

--- last ${LOG_TAIL} log lines ---
$(docker logs --tail "$LOG_TAIL" "$c" 2>&1)"

    elif [ "$status" = "running" ] && [ "$prev_status" = "restarting" ]; then
        subject="[RECOVERED] UAT ${c} running again on ${HOST}"
        body="Time          : $(date -Is)
Recovered from : crash loop
Total restarts : ${count}"
    fi
    # exited (manual stop) / missing / steady state -> silent

    if [ -n "$subject" ]; then
        send_mail "$subject" "$body" && echo "${status}|${count}" > "$sf"
    else
        echo "${status}|${count}" > "$sf"
    fi
fi
}

```

```
for c in "${CONTAINERS[@]}"; do
    check_container "$c"
done
exit 0
```

## Step 5 — Permissions

```
bash

chown root:root /opt/monitoring/container-monitor.sh /etc/container-monitor.conf
chmod 750 /opt/monitoring/container-monitor.sh      # root exec only
chmod 640 /etc/container-monitor.conf              # root read only
bash -n /opt/monitoring/container-monitor.sh && echo "syntax OK" # paste sanitized
```

File layout reference:

| File   | Path                                                       | Purpose                                                          |
|--------|------------------------------------------------------------|------------------------------------------------------------------|
| script | <code>/opt/monitoring/container-monitor.sh</code>          | logic                                                            |
| config | <code>/etc/container-monitor.conf</code>                   | per-server settings                                              |
| state  | <code>/var/lib/container-monitor/&lt;name&gt;.state</code> | auto-created, script memory<br>( <code>(status   count)</code> ) |
| log    | <code>/var/log/container-monitor.log</code>                | via cron redirect                                                |

**Alternative to Steps 3-4:** copy from an already-installed server (`scp /opt/monitoring/container-monitor.sh /etc/container-monitor.conf user@target:/tmp/` as root, then `mv` into place). Use the **internal** hostname/IP (`(* .nhorbcomm.internal)`, `(10.216.x.x)`) — external FQDNs may resolve to unreachable addresses. Still edit CONTAINERS and the env label after copying.

## Step 6 — SMTP smoke test (per server — relay may whitelist by source IP)

```
bash

printf 'From: no-reply@nhorbcomm.internal\r\nTo: YOU@company.com\r\nSubject: sm
curl -v --url smtp://smtp.nhorbcomm.internal:587 \
    --mail-from no-reply@nhorbcomm.internal --mail-rcpt YOU@company.com --uplo
```

Watch for `250 Accepted` after RCPT TO and check the inbox. ⚠️ The **envelope recipient** (`-mail-rcpt`) determines delivery, not the `To:` header.

## Step 7 — First run

```
bash

/opt/monitoring/container-monitor.sh; echo "exit: $?"
cat /var/lib/container-monitor/<container>.state    # expect: running|0
```

Silence + exit 0 = healthy baseline. A `[FATAL]` line = config problem; the message says exactly what. Config changes never need any reload — the script re-reads the conf on every run.

## Step 8 — Cron (editor-free, idempotent)

```
bash

crontab -l 2>/dev/null | grep -q container-monitor.sh || \
(crontab -l 2>/dev/null; echo '*/*5 * * * * /opt/monitoring/container-monitor.sh')
crontab -l | tail -2                                # verify
```

Confirm heartbeat after ~6 minutes:

```
bash

ls -l --time-style=full-iso /var/lib/container-monitor/    # mtime advances every
```

## Step 9 — Log rotation

```
bash

tee /etc/logrotate.d/container-monitor << 'EOF'
/var/log/container-monitor.log {
    weekly
    rotate 8
    compress
    missingok
    notifempty
}
EOF
```

## 5. Verification tests (minimum per new server)

### T1 — Crash → [CRASH] mail (proves Layer 1 + Layer 2 + mail end to end)

```
bash
kill -9 $(docker inspect -f '{{.State.Pid}}' <container>) # HOST-level kill =
sleep 5
docker ps --filter name=<container> # must ALREADY be Up ← Layer 1 wo
/opt/monitoring/container-monitor.sh # → [INFO] Mail sent: [CRASH] ...
```

⚠ Must be `kill -9 <PID>` from the host. `docker kill` goes through the Docker API → treated as a manual stop → no restart, no mail (correct behavior, wrong test).

### T2 — Manual stop/start → total silence (proves the headline rule)

```
bash
docker stop <container>
/opt/monitoring/container-monitor.sh # no mail, state: exited|N
docker start <container>
/opt/monitoring/container-monitor.sh # no mail, state: running|N
```

### T3 — Crash loop → [DOWN] then [RECOVERED] (optional; causes real downtime)

```
bash
CFG=$(docker inspect <container> --format '{{range .Mounts}}{{.Source}}{{"\n"}}')
cp "$CFG" "$CFG.bak"
echo "{ this is not valid json" > "$CFG" # corrupt CONTENT — do NOT delete/
docker restart <container>
sleep 30
docker inspect -f '{{.State.Status}}' <container> # restarting
/opt/monitoring/container-monitor.sh # → [DOWN] mail
/opt/monitoring/container-monitor.sh # → silence (no spam)
cp "$CFG.bak" "$CFG" # heal
sleep 70 # let Docker's own retry su
/opt/monitoring/container-monitor.sh # → [RECOVERED] mail
rm "$CFG.bak"
```

## 6. Alert reference

| Subject                                                                               | Meaning                                         | Action needed                                                                                                          |
|---------------------------------------------------------------------------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>[CRASH] &lt;env&gt; &lt;c&gt;</code><br><code>crashed and auto-recovered</code> | crashed, already restarted by Docker            | investigate cause from attached logs; no urgency — service is up. Rising RestartCount across mails = recurring problem |
| <code>[DOWN] &lt;env&gt; &lt;c&gt; is crash-looping</code>                            | failing on every start; Docker retrying forever | <b>act now</b> — usually bad config or bad deploy. Fix root cause; loop self-resolves on next successful start         |
| <code>[RECOVERED] &lt;env&gt; &lt;c&gt;</code><br><code>running again</code>          | crash loop ended                                | confirm in Grafana that consumer lag is draining                                                                       |
| <code>[CRITICAL] &lt;env&gt;</code><br><code>Docker daemon down</code>                | dockerd not responding — everything unknown     | <b>act now</b> — <code>systemctl status docker</code> on the host                                                      |

Mail body fields: `OOMKilled: true` → memory-limit problem (capacity); `false` → application exception (code/config). `RestartCount: X -> Y` with delta >1 = multiple crashes within one 5-minute window.

## 7. Troubleshooting / lessons learned

### 7.1 Mail fails with `curl: (8) Weird server reply`

Cause: SMTP requires CRLF line endings; docker log lines are LF. The script pipes the whole message through `sed -e 's/\r$//' -e 's/^\n/./' -e 's/$/\n/'` before curl. If mails fail, verify that pipeline is present in `send_mail()`. Failed mails are auto-retried (state file only advances on successful send).

### 7.2 Crash test shows no restart / no mail

Almost always: you used `docker kill` (API = manual) instead of host-level `kill -9 <PID>`, or the restart policy regressed to `no` after a redeploy (check `docker inspect -f '{{.HostConfig.RestartPolicy.Name}}'`). The silent-on-`exited` rule is only valid while `unless-stopped` is guaranteed.

### 7.3 Never delete/rename a bind-mounted config file for testing

If the mount source is missing at container start, Docker creates a directory in its place →

container cannot start at all (mount error, not a crash loop) and the directory must be `rm -rf`'d before restoring the file. Corrupt the file's *content* instead (§5 T3).

#### 7.4 No mail for a real crash — is it lost?

No. Check `/var/log/container-monitor.log` for `Mail FAILED` and the state file: if it still shows the pre-crash snapshot, the event is pending and retried every 5 min until a send succeeds.

#### 7.5 vim opened on `crontab -e`

Use `EDITOR=nano crontab -e`, or the editor-free append in Step 8. If the terminal prints mouse-coordinate garbage afterwards: `reset`.

#### 7.6 scp between servers hangs at "Connecting to..."

Use the **internal** hostname (`fe4-docker-XX.nhorbcomm.internal`) or the `10.216.x.x` IP. External FQDNs (`*.tms.orbcomm.com`) can resolve to a `192.168.x` address that is unreachable/firewalled between these servers.

#### 7.7 Config file permission hardening (`chmod 600`)

Safe on a running container — bind-mounted file permissions apply instantly, no restart needed, and the app read its config at startup anyway. BUT verify the container's runtime user can still read it (matters at NEXT start):

```
bash

docker exec <container> id                                # uid=0 → saf
docker exec <container> cat /app/Config/appsettings.*.json >/dev/null && echo R
```

#### 7.8 Monitored container reports "missing" permanently

The container was likely **renamed or removed by a deploy** — `missing` is silent by design (covers the mid-deploy window), so a rename means permanent silent non-monitoring. After any deploy that changes container names, update `CONTAINERS=(...)` in the conf and delete the stale state file.

### 8. Known limitations (accepted scope)

| Gap               | Why                                                            | Mitigation / future work                                                                       |
|-------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Whole server down | script dies with the server; self-monitoring cannot report its | Future: <code>node_exporter</code> + Prometheus <code>up == 0</code> rule (external dead man's |

| Gap                                                     | Why                                                                                                    | Mitigation / future work                                                                  |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
|                                                         | own death                                                                                              | switch), or Healthchecks.io heartbeat.<br>Optional <code>@reboot</code> notification mail |
| <b>Hung container</b><br>(process alive, doing nothing) | Docker sees it "running"; no exit<br>→ no restart, no count change                                     | Grafana Kafka consumer-lag alert is the hang detector for the bridge                      |
| <b>Manual stop forgotten</b>                            | silent by explicit team decision                                                                       | team convention: announce maintenance stops                                               |
| <b>Container renamed/removed</b>                        | <code>missing</code> is silent                                                                         | update conf after deploys (§7.8)                                                          |
| <b>Slow manual loop fix</b><br>(stop >5 min mid-fix)    | intermediate <code>exited</code> observation breaks the restarting→running chain → no [RECOVERED] mail | cosmetic only; the person fixing knows the outcome                                        |
| Detection latency up to 5 min                           | cron interval                                                                                          | acceptable; Layer 1 already fixed the outage in ~1s                                       |
| Env label hardcoded in script                           | four <code>subject=</code> lines say <code>UAT</code>                                                  | edit per server (Step 4 note)                                                             |

## 9. Server rollout checklist

- ☐ `docker update --restart unless-stopped <each app container>`
- ☐ verify: policy = `unless-stopped` for all monitored containers
- ☐ `systemctl is-enabled docker` → enabled
- ☐ config created at `/etc/container-monitor.conf` — CONTAINERS = **this** server's names
- ☐ script created at `/opt/monitoring/container-monitor.sh` — env label edited (UAT/PROD/QA)
- ☐ permissions: script 750 root, conf 640 root; `bash -n` passes
- ☐ SMTP smoke test **from this server**
- ☐ first manual run → silence, state files `running|N`
- ☐ cron installed, `crontab -l` verified
- ☐ heartbeat: state-file mtime advances after 5-10 min
- ☐ T1 crash test → [CRASH] mail received

- ☐ T2 stop/start → silence
- ☐ logrotate config in place
- ☐ CI/CD deploy script for this server's containers carries `--restart unless-stopped`